

train_test_split

$X = [\blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare] (100\%)$
 $y = [\text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯}] (100\%)$



▼ `train\_test\_split(X, y, test\_size=0.2)` 실행!



$X_{\text{train}} = [\blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare] (80\%)$
 $y_{\text{train}} = [\text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯}] (80\%)$

훈련용
데이터

$X_{\text{test}} : [\blacksquare \blacksquare] (20\%)$
 $y_{\text{test}} : [\text{🎯} \text{🎯}] (20\%)$

검증용
데이터

StandardScaler

역할: 데이터의 단위(스케일)를 맞춰서 공평하게 만들어주는 도구

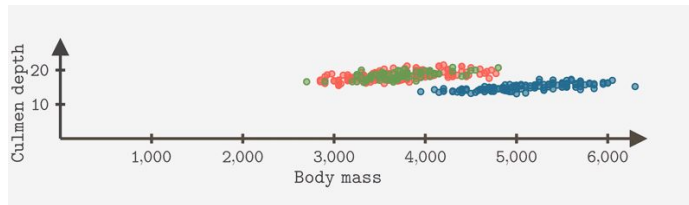
💡 왜 필요할까?

만약 사람의 건강 데이터를 가지고 질병을 예측하는데, 어떤 데이터는 '키(cm)'이고 어떤 데이터는 '몸무게(kg)'라고 가정.

- 키는 대략 150 ~ 190 사이의 값
- 몸무게는 40 ~ 100 사이의 값.

만약 키 170, 몸무게 70인 데이터에서
컴퓨터는 숫자의 크기만 보고 "와, 키는 170이니까 몸무게보다 2.5배 더 중요한 데이터구나 !" 하고 판단.

이런 오해를 막기 위해 모든 데이터의 평균을 0, 표준편차를 1로 만들어서 치우침 없이 공평한 저울 위에 올려놓는 작업(표준화)을 수행



StandardScaler 더 알아보기

<https://codefinity.com/courses/v2/a65bbc96-309e-4df9-a790-a1eb8c815a1c/1fce4aa9-710f-4bc9-ad66-16b4b2d30929/79d587a4-bba9-45c8-878f-f2948f0b0c7e>

DecisionTreeClassifier

역할: 스무고개처럼 질문을 던지며 정답을 맞춰가는 분류 모델

💡 왜 필요할까?

데이터를 다 고르고 나면 컴퓨터가 학습 필요

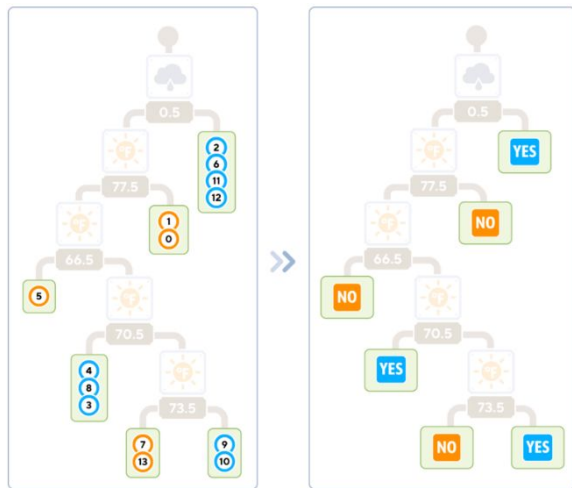
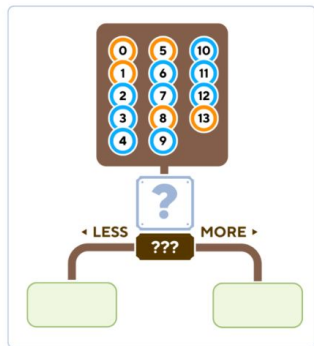
DecisionTreeClassifier는 데이터를 학습해서 스무고개 지도(Tree)를 그리는 인공지능 모델.

예를 들어, "이 동물이 타이거(호랑이)일까, 고양이일까?"를 분류할 때 컴퓨터는 아래와 같이 판단을 수행.

1. "몸무게가 50kg 이상인가요?" → Yes / No
2. (Yes인 경우) "줄무늬가 있나요?" → Yes (호랑이!) / No (사자!)

Training Set

0	...	NO
1	...	NO
2	...	YES
3	...	YES
4	...	YES
5	...	NO
6	...	YES
7	...	YES
8	...	NO
9	...	YES
10	...	YES
11	...	YES
12	...	YES
13	...	NO



DecisionTreeClassifier 더 알아보기

<https://towardsdatascience.com/decision-tree-classifier-explained-a-visual-guide-with-code-examples-for-beginners-7c863f06a71e/>

Pipeline

역할 : 머신러닝의 '자동화 컨베이어 벨트'.

앞에서 배운 데이터 전처리(**StandardScaler**)와 모델 학습(**DecisionTreeClassifier**) 과정을 하나의 매끄러운 파이프라인으로 묶어서 한 번에 실행해 주는 도구.

만약 파이프라인을 사용한다면, 선행 작업(**StandardScaler**)이 끝났는지 판단하고 다음 작업(**DecisionTreeClassifier**)을 실행해주는 과정을 코드로 직접 작성해줘야 함.

```
# 1. 스케일러 따로 학습 및 변환
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 2. 모델 따로 학습 및 평가
model = DecisionTreeClassifier()
model.fit(X_train_scaled, y_train)
accuracy = model.score(X_test_scaled, y_test)
```

```
from sklearn.pipeline import Pipeline

# 컨베이어 벨트 조립: 1번 공정(scaler) -> 2번 공정(tree)
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('tree', DecisionTreeClassifier())
])

# 학습 데이터만 집어넣으면 스케일링과 모델 학습이 원터치로 진행!
pipeline.fit(X_train, y_train)
```

```
from sklearn.pipeline import Pipeline
# 1. 내가 원하는 다른 도구들을 불러온다
from sklearn.preprocessing import MinMaxScaler # 다른 종류의 스케일러
from sklearn.ensemble import RandomForestClassifier # 다른 종류의 모델

# 2. 내 입맛대로 컨베이어 벨트를 조립한다
my_pipeline = Pipeline([
    ('my_scaler', MinMaxScaler()), # 내가 고른 스케일러
    ('my_model', RandomForestClassifier()) # 내가 고른 모델
])
```

공정 단계	도구 (예시)	언제 쓸까?
1단계: 데이터 전처리 (Scaler, Imputer 등)	StandardScaler MinMaxScaler SimpleImputer	데이터의 단위를 맞출 때 데이터가 0~1 사이에 있어야 할 때 비어있는 데이터(결측치)를 채울 때
2단계: 예측 모델 (Classifier, Regressor 등)	DecisionTreeClassifier LinearRegression SVC (Support Vector Machine)	스무고개 형식으로 분류할 때 집값이나 주가 같은 연속된 숫자를 예측할 때 비선형 데이터를 분류할 때

SimpleImputer

역할: 비어있는 값(결측치, NaN)을 자동으로 채워주는 역할

💡 왜 필요할까?

현실에서 데이터를 수집하다 보면 설문조사에서 누락된 답변이 있거나, 기기 오류로 인해 값이 입력되지 않아 데이터가 텅 비어 있는 경우

빈 칸(NaN)이 있으면 연산을 못 하고 에러를 발생시킴.

사용자가 정한 규칙(전략)에 따라 빈 칸을 채울 수 있음. 주로 아래의 4가지 전략(strategy)을 사용

1. **strategy='mean' (평균값):** 나머지 숫자들의 평균으로 채움.
2. **strategy='median' (중앙값):** 데이터를 크기순으로 세웠을 때 가운데 있는 값으로 채움.
3. **strategy='most_frequent' (최빈값):** 가장 자주 등장한 값으로 채움.
"성별(남/여)"이나 "혈액형" 같은 글자(범주형) 데이터의 빈칸을 채울 때 사용.
4. **strategy='constant' (상수):** 사용자가 지정한 특정 숫자나 글자(예: `fill_value=0` 또는 `'Missing'`)로 똑같이 채움.

SimpleImputer 더 알아보기

<https://m.blog.naver.com/youji4ever/222137468851>

<https://www.youtube.com/watch?v=GFnwwZVCMro>

ColumnTransformer

💡 왜 필요할까?

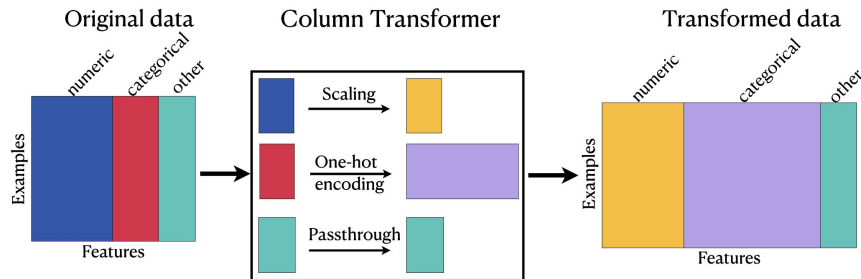
현실의 데이터셋은 보통 **숫자 데이터** (나이, 자산 등)와 **글자(범주형) 데이터** (성별, 거주지역 등)가 섞여 있음.

- 숫자 데이터에는 `SimpleImputer(strategy='mean')`로 평균을 채우고, `StandardScaler`를 적용.
- 글자 데이터에는 평균을 낼 수 없음. `SimpleImputer(strategy='most_frequent')`로 최빈값을 채우고, 숫자로 바꿔주는 변환(`OneHotEncoder` 등)을 해야 함.

컬럼의 종류(숫자냐 글자냐)에 따라 데이터를 여러 라인으로 쪼개서 각각 다른 공정을 적용한 뒤, 마지막에 다시 하나로 합쳐줌.

🔗 Pipeline vs ColumnTransformer 비교

- **Pipeline (직렬 처리):** 하나의 라인에서 순서대로 공정을 진행. (예: 빈칸 채우기 → 단위 맞추기)
- **ColumnTransformer (병렬 처리):** 컬럼의 종류(숫자 or 글자)에 따라 데이터를 여러 라인으로 쪼개서 각각 다른 공정을 적용



Columntransformer 더 알아보기

<https://bait509-ubc.github.io/BAIT509/lectures/lecture5.html#columntransformer>

OneHotEncoder

역할: 글자(범주형) 데이터를 컴퓨터가 이해할 수 있는 이진수(0과 1)의 형태로 완벽하게 바꿔주는 번역기

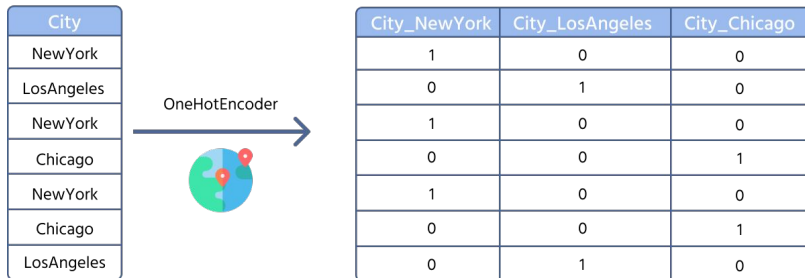
💡 왜 필요할까?

컴퓨터, 특히 머신러닝 모델은 수학 연산을 기반으로 움직이기 때문에 '남자', '여자' 같은 글자나 '서울', '부산', '제주' 같은 지역 이름을

직접 계산식에 넣을 수 없기 때문에 이 글자들을 컴퓨터가 읽을 수 있는 숫자 데이터로 변환

만약 우리 데이터에 [NewYork, LosAngeles, Chicago]라는 3가지 지역이 있다면,
이 종류(범주)만큼 새로운 열(Column)을 만들고, 해당하는 자리에만 1을, 나머지 자리에는 0을 채워 넣는 방식

Nominal



OneHotEncoder 더 알아보기

<https://codefinity.com/courses/v2/a65bbc96-309e-4df9-a790-a1eb8c815a1c/1fce4aa9-710f-4bc9-ad66-16b4b2d30929/a6d33d0d-3057-4a2f-b8df-4ecd00ffd598>

KFold

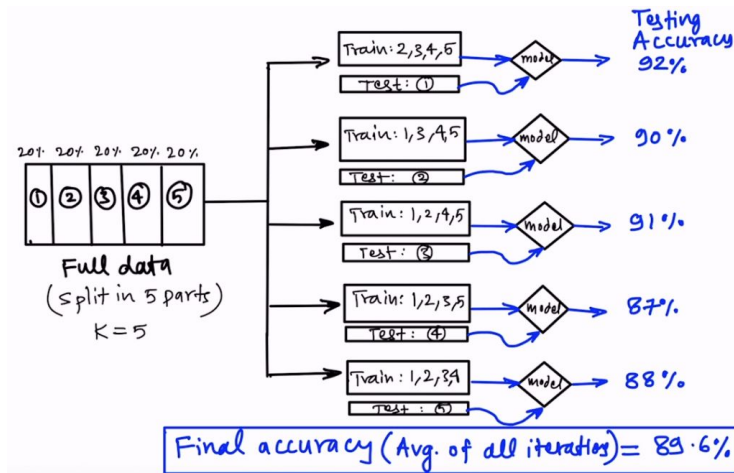
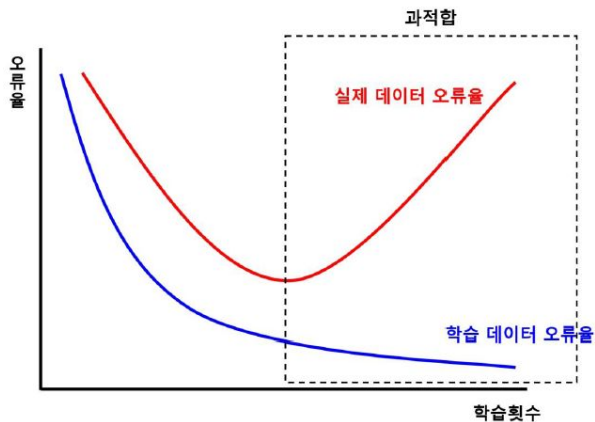
역할: 데이터를 K개의 방으로 쪼개서, 돌아가며 시험(Test)을 진행하게 해주는 역할.

💡 왜 필요할까?

`train_test_split`으로 데이터를 8:2로 딱 한 번만 나누면, 우연히 테스트 데이터(2)에 너무 쉬운 문제만 들어가거나 반대로 너무 어려운 문제만 들어갈 수 있음. (과적합, 편향문제)

KFold는 데이터를 K개(예: 5개)의 덩어리(Fold)로 똑같이 쪼개 뒤, 모든 데이터가 한 번씩은 시험 문제(Validation)가 되도록 K번 굴리는

방식



KFold 더 알아보기

<https://ddukbbok-kang.tistory.com/30>

cross_validate

역할: KFold가 쪼개준 시험 (Test)으로 채점하고 성적표를 뽑아줌

💡 왜 필요할까?

KFold는 데이터를 어떻게 쪼갤지 '방향'만 잡아주는 도구일 뿐,
실제로 그 쪼개진 데이터로 모델을 학습시키고 점수를 계산하는 건 **for**문 코드를 짜서 일일이 돌려야 함.

이 번거로운 반복 작업을 **함수 한 줄로 알아서 실행하고 성적표까지 예쁘게 리턴**해주는 도구가 바로 **cross_validate**.

성적표 항목 (Key)	의미
<code>fit_time</code>	각 회차별로 모델을 학습하는 데 걸린 시간 (초)
<code>score_time</code>	각 회차별로 모델을 채점(예측)하는 데 걸린 시간 (초)
<code>test_score</code>	우리가 궁금한 테스트(검증) 점수 → 이 5개 점수의 평균이 모델의 진짜 실력!
<code>train_score</code>	공부할 때 쓴 문제로 다시 시험 본 점수 (옵션 설정 시 확인 가능)

cross_validate

cv_results['train_rmse'] 같은건 뭔가?

```
# 2. 평가 지표(Scoring) 디렉터리 정의
# 사이킷런의 채점 시스템은 점수가 높을수록 좋은 것으로 인식합니다.
# 따라서 오차값(Error)인 RMSE와 MAE는 내부적으로 음수(neg_)로 계산되도록 이름이 정해져 있습니다.
scoring = {
    'rmse': 'neg_root_mean_squared_error', # 평균 제곱근 오차 (실제값과 예측값 차이)
    'mae': 'neg_mean_absolute_error',      # 평균 절대 오차
    'r2': 'r2',                            # 결정계수 (1에 가까울수록 성능 좋음, 이걸 원래 클수록 좋으므로 neg_가 안 붙음)
}
```

scoring 변수에 넣어준 디렉터리에서 정의됨 (채점할 기준을 정의)

이 변수를 cross_validate 함수의 scoring 매개변수에 넣어줬기 때문에 뽑아낼 수 있음.

```
# 3. 교차 검증 실행
cv_results = cross_validate(
    pipeline,          # 앞에서 만든 전처리+모델 결합 파이프라인
    X_train_ready,     # 파생변수와 바이닝 처리가 완료된 학습용 독립변수 데이터
    y_train,           # 예측해야 할 정답 (집값 데이터)
    cv=cv,             # 위에서 정의한 5-Fold 분할 규칙
    scoring=scoring,   # 채점할 기준 3가지
    return_train_score=True, # 평가용 폴드의 점수뿐만 아니라, 학습에 썼던 폴드의 점수도 같이 반환할지 여부
)
```

cross_validate

cv_results['train_rmse'] 같은건 뭔가?

```
# 2. 평가 지표(Scoring) 디렉터리 정의
# 사이킷런의 채점 시스템은 점수가 높을수록 좋은 것으로 인식합니다.
# 따라서 오차값(Error)인 RMSE와 MAE는 내부적으로 음수(neg_)로 계산되도록 이름이 정해져 있습니다.
scoring = {
    'rmse': 'neg_root_mean_squared_error', # 평균 제곱근 오차 (실제값과 예측값 차이)
    'mae': 'neg_mean_absolute_error',      # 평균 절대 오차
    'r2': 'r2',                           # 결정계수 (1에 가까울수록 성능 좋음, 이걸 원래 클수록 좋으므로 neg_가 안 붙음)
}
```

scoring 변수에 넣어준 디렉터리에서 정의됨 (채점할 기준을 정의)

이 변수를 cross_validate 함수의 scoring 매개변수에 넣어줬기 때문에 뽑아낼 수 있음.

```
# 3. 교차 검증 실행
cv_results = cross_validate(
    pipeline,          # 앞에서 만든 전처리+모델 결합 파이프라인
    X_train_ready,     # 파생변수와 바이닝 처리가 완료된 학습용 독립변수 데이터
    y_train,           # 예측해야 할 정답 (집값 데이터)
    cv=cv,             # 위에서 정의한 5-Fold 분할 규칙
    scoring=scoring,   # 채점할 기준 3가지
    return_train_score=True, # 평가용 폴드의 점수뿐만 아니라, 학습에 썼던 폴드의 점수도 같이 반환할지 여부
)
```

scoring option 더 알아보기

https://scikit-learn.org/stable/modules/model_evaluation.html#scoring-parameter